

01-406-093-203

การพัฒนาเว็บแอปพลิเคชันด้วยเฟรมเวิร์ค

โดย นายศุภมิตร จันทรโทวงศ์

- สาขาวิชาระบบสารสนเทศ คณะบริหารธุรกิจ

เช็คชื่อเข้าชั้นเรียน

- ให้นักศึกษาสแกน QR Code นี้เพื่อเข้าสู่ระบบเช็คชื่อเข้าเรียน
- ระบบจะทำการบันทึกประวัติการเข้าเรียนในวิชานี้อัตโนมัติ

สแกนเพื่อเช็คชื่อเข้าชั้นเรียน (สัปดาห์ที่ 13)



สแกน QR Code เพื่อเช็คชื่อเข้าเรียนของสัปดาห์นี้

แบบทดสอบก่อนเรียน (Pre-test)

1. การดึงข้อมูลทั้งหมดแบบแบ่งหน้า (Pagination) ผ่าน Eloquent Model ใน Laravel ควรเขียนคำสั่งอย่างไร?

A. `Blog::paginate(5);`

B. `Blog::all()->paginate(5);`

C. `DB::paginate('blogs', 5);`

D. `Blog::get()->page(5);`

3. การลบข้อมูลบทความตามรหัส ID ผ่านคำสั่ง Eloquent Model ข้อใดถูกต้อง?

A. `Blog::find($id)->delete();`

B. `Blog::delete($id);`

C. `Blog::remove($id);`

D. `Blog::drop($id);`

5. ตัวแปรใดใน Model ของ Laravel ที่ใช้สำหรับระบุชื่อฟิลด์คอลัมน์ที่อนุญาตให้เขียนหรืออัปเดตข้อมูลได้?

A. `protected $fillable = [...];`

B. `protected $allow = [...];`

C. `protected $columns = [...];`

D. `protected $whiteList = [...];`

7. หากต้องการจัดกลุ่มเส้นทาง (Routing) ให้มี URL นำหน้าด้วยคำว่า /author เหมือนกันทั้งหมด ต้องใช้คำสั่งใดใน web.php?

A. `Route::prefix('author')->group(function () { ... });`

B. `Route::group('author', function () { ... });`

C. `Route::name('author')->prefix(function () { ... });`

D. `Route::path('author')->group(function () { ... });`

9. ในแบบฟอร์มเขียนบทความ (form.blade.php) หากจัดกลุ่ม Route ให้อยู่ที่ /author แล้ว ค่าของ action ในแท็กฟอร์มควรเป็นข้อใด?

A. `<form method="POST" action="/author/insert">`

B. `<form method="POST" action="/insert">`

C. `<form method="GET" action="/author/save">`

D. `<form method="POST" action="/admin/post">`

2. หากต้องการค้นหาข้อมูลบทความตามรหัส ID เพื่อนำมาแสดงหรือแก้ไขผ่าน Eloquent Model ต้องใช้คำสั่งใด?

A. `Blog::find($id);`

B. `Blog::search($id);`

C. `Blog::selectId($id);`

D. `Blog::getById($id);`

4. เมื่อเกิดข้อผิดพลาด MassAssignmentException สาเหตุหลักเกิดจากอะไร?

A. ยังไม่ได้กำหนดตัวแปร \$fillable หรือ \$guarded ใน Model เพื่ออนุญาตให้บันทึกข้อมูลฟิลด์นั้น

B. ฐานข้อมูลไม่รองรับคำสั่ง SQL แบบ ORM

C. ไม่ได้เชื่อมต่อ Database ในไฟล์ .env

D. ชื่อตารางในฐานข้อมูลไม่ตรงกับชื่อ Model

6. หากต้องการสั่งให้อัปเดตข้อมูลบทความตามรหัส ID ผ่าน Model ต้องเขียนคำสั่งอย่างไร?

A. `Blog::find($id)->update($data);`

B. `Blog::update($id, $data);`

C. `Blog::edit($id)->save($data);`

D. `Blog::set($data)->where($id);`

8. หลังจากบันทึกบทความใหม่ใน Controller สำเร็จ หากต้องการเปลี่ยนเส้นทางไปยังหน้ารายการบทความของนักเขียน (/author/blog) ต้องใช้คำสั่งใด?

A. `return redirect('/author/blog');`

B. `return route('/author/blog');`

C. `return view('/author/blog');`

D. `return go('/author/blog');`

10. หากต้องการให้หลังจากลบข้อมูลหรือสลับสถานะแล้ว ให้เบราว์เซอร์เปลี่ยนเส้นทางกลับไปยังหน้าเดิมของผู้ใช้ ควรใช้คำสั่งใด?

A. `return redirect()->back();`

B. `return redirect()->previous();`

C. `return back()->redirect();`

D. `return route('back');`

การจัดการข้อมูลระดับโมเดล (Eloquent Model & Mass Assignment)

Eloquent & Mass Assignment

- Mass Assignment คือความสามารถในการบันทึกข้อมูลหลายฟิลด์เข้าสู่โมเดลในครั้งเดียวด้วยคำสั่ง create()
- หากไม่มีการป้องกัน ผู้ไม่หวังดีอาจใช้วิธีลักไก่แก้ฟอร์มเพื่อส่งค่าฟิลด์ที่ไม่พึงประสงค์เข้ามาเขียนกับ (เช่น is_admin = true)
- Laravel บังคับให้นักพัฒนาต้องเลือกช่องทางการกรอกข้อมูลให้ปลอดภัยผ่านตัวแปรระดับโมเดล
- ตัวแปรป้องกัน: \$fillable (อาเรียระบุฟิลด์ที่อนุญาต) หรือ \$guarded (ฟิลด์ห้ามบันทึกโดยตรง)

code-example.php

```
// app/Models/Book.php
class Book extends Model
{
    // กำหนดฟิลด์ที่สามารถกรอกผ่านคำสั่ง Book::create() ได้อย่างปลอดภัย
    protected $fillable = ['title', 'author', 'price'];
}
```

การเรียกใช้ข้อมูล: Query Builder vs Eloquent Model

- มีข้อมูลอยู่แล้ว จะเรียกใช้ผ่าน Query Builder ไม่ผ่าน Model
- Model > SQL ดำเนินการกับ Model โดยตรง
- Eloquent

การเรียกใช้งาน Model ใน Controller (AdminController.php)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- นำเข้า Model โดยเพิ่มคำสั่ง use App\Models\Blog; ที่ด้านบนของไฟล์
- ดึงข้อมูลผ่าน Model โดยตรงด้วยคำสั่ง \$blogs = Blog::paginate(5); แทนการใช้ Query Builder
- ส่งข้อมูล \$blogs ไปแสดงผลที่หน้า View ด้วยคำสั่ง compact('blogs')

code-example.php

```
// app/Http/Controllers/AdminController.php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Blog;

class AdminController extends Controller
{
    public function __construct()
    {
        $this->middleware('auth');
    }

    function blog()
    {
```

สัปดาห์ที่ 13 · การจัดการข้อมูลระดับโมเดลและคำนำหน้าเส้นทาง

01-406-093-203 LARAVEL COURSE

การจัดการข้อมูลบทความผ่าน Eloquent Model

อาจารย์ผู้บรรยาย: ศุภมิตร จันทรวงศ์

สไลด์ที่ 8 / 32

การบันทึกข้อมูลผ่าน Model (ฟังก์ชัน insert)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- ตรวจสอบความถูกต้องของข้อมูลด้วยฟังก์ชัน \$request->validate()
- จัดเตรียมข้อมูลให้อยู่ในรูปแบบ Array (\$data)
- บันทึกข้อมูลลงตารางผ่าน Model โดยตรงด้วยคำสั่ง Blog::insert(\$data);
- เปลี่ยนเส้นทางกลับไปยังหน้าบล็อกด้วย return redirect('/blog');

code-example.php

```
// app/Http/Controllers/AdminController.php
function insert(Request $request)
{
    $request->validate([
        'title' => 'required|max:50',
        'content' => 'required',
    ], [
        'title.required' => 'กรุณาใส่ชื่อบทความ',
        'title.max' => 'ชื่อบทความต้องไม่เกิน 50 ตัวอักษร',
        'content.required' => 'กรุณาใส่เนื้อหา',
    ])
};

$data = [
```

การลบข้อมูลผ่าน Model (ฟังก์ชัน delete)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- รับค่ารหัสข้อมูล (\$id) ที่ส่งเข้ามาในฟังก์ชัน delete
- ค้นหาข้อมูลและส่งลบผ่าน Model ในบรรทัดเดียวด้วยคำสั่ง `Blog::find($id)->delete();`
- เมื่อลบข้อมูลเสร็จเรียบร้อย ให้เปลี่ยนเส้นทางกลับไปยังหน้าบล็อกด้วย `return redirect('/blog');`

code-example.php

```
// app/Http/Controllers/AdminController.php
function delete($id)
{
    Blog::find($id)->delete();
    return redirect('/blog');
}
```

การเปลี่ยนเส้นทางกลับหน้าเดิม (redirect()->back())

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- ปรับปรุงการเปลี่ยนเส้นทางในฟังก์ชัน delete
- ใช้คำสั่ง return redirect()->back(); เพื่อพากลับไปยังหน้าที่ส่งคำขอมาโดยตรง
- ช่วยให้ผู้ใช้ยังคงอยู่หน้าเดิมที่กำลังทำงาน เช่น หน้าตารางที่มี Pagination

code-example.php

```
// app/Http/Controllers/AdminController.php
function delete($id)
{
    Blog::find($id)->delete();
    return redirect()->back();
}
```

การค้นหาข้อมูลเพื่อแก้ไขผ่าน Model (ฟังก์ชัน edit)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- รับค่ารหัสข้อมูล (\$id) ในฟังก์ชัน edit เพื่อใช้ค้นหาบทความที่ต้องการแก้ไข
- ค้นหาข้อมูลตามรหัสผ่าน Model ด้วยคำสั่ง `$blog = Blog::find($id);` แทนการใช้ `DB::table`
- ส่งข้อมูล `$blog` ไปยังหน้าฟอร์มแก้ไขด้วย `return view("edit", compact('blog'));`

code-example.php

```
// app/Http/Controllers/AdminController.php
function edit($id)
{
    $blog = Blog::find($id);
    return view("edit", compact('blog'));
}
```

การอัปเดตข้อมูลผ่าน Model (ฟังก์ชัน update)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- ตรวจสอบความถูกต้องของข้อมูลที่ส่งมาแก้ไขด้วยฟังก์ชัน \$request->validate()
- จัดเตรียมข้อมูลที่ต้องการแก้ไขให้อยู่ในรูปแบบ Array (\$data)
- ค้นหาข้อมูลตามรหัสและสั่งอัปเดตผ่าน Model ด้วยคำสั่ง Blog::find(\$id)->update(\$data);
- เมื่ออัปเดตสำเร็จ ให้เปลี่ยนเส้นทางกลับไปยังหน้าบล็อกด้วย return redirect('/blog');

code-example.php

```
// app/Http/Controllers/AdminController.php
function update(Request $request, $id)
{
    $request->validate([
        'title' => 'required|max:50',
        'content' => 'required',
    ], [
        'title.required' => 'กรุณาใส่ชื่อบทความ',
        'title.max' => 'ชื่อบทความต้องไม่เกิน 50 ตัวอักษร',
        'content.required' => 'กรุณาใส่เนื้อหา',
    ])
};

$data = [
```

ข้อผิดพลาด MassAssignmentException จากการอัปเดตข้อมูล

- เมื่อสั่งอัปเดตข้อมูลผ่าน Model ด้วยคำสั่ง `Blog::find($id)->update($data);`
- ระบบจะเกิดข้อผิดพลาด 500 Internal Server Error: MassAssignmentException
- สาเหตุ: Laravel ป้องกันไม่ให้เขียนข้อมูลลงฟิลด์โดยตรงหากยังไม่ได้อนุญาตใน Model
- ข้อความแจ้งเตือน: Add [title] to fillable property to allow mass assignment on [App\Models\Blog]
- แนวทางแก้ไข: ต้องเข้าไปกำหนดตัวแปร `$fillable` ภายในไฟล์ Model (Blog.php)

code-example.bash

```
Illuminate\Database\Eloquent\MassAssignmentException
```

```
Add [title] to fillable property to allow mass assignment on [App\Models\Blog].
```

```
// Endpoint ที่เรียกใช้งาน
```

```
POST http://127.0.0.1:8000/update,41
```

การกำหนด \$fillable ใน Model (Blog.php)

- เปิดไฟล์โมเดล app/Models/Blog.php
- ประกาศตัวแปร protected \$fillable ภายในคลาส Blog
- ระบุรายชื่อฟิลด์คอลัมน์ที่อนุญาตให้บันทึกหรือแก้ไขข้อมูล ได้แก่ title, content และ status
- ช่วยป้องกันข้อผิดพลาด MassAssignmentException และทำให้สามารถบันทึกข้อมูลแบบแถวเดียวได้อย่างปลอดภัย

code-example.php

```
// app/Models/Blog.php
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Blog extends Model
{
    use HasFactory;

    protected $fillable = [
        'title',
        'content',
        'status'
    ];
}
```

การสลับสถานะข้อมูลผ่าน Model (ฟังก์ชัน change)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- รับค่ารหัสข้อมูล (\$id) ในฟังก์ชัน change
- ค้นหาข้อมูลเดิมด้วยคำสั่ง \$blog = Blog::find(\$id); เพื่อตรวจสอบสถานะปัจจุบัน
- สลับค่าสถานะเป็นตรงกันข้ามด้วย 'status' => !\$blog->status
- อัปเดตข้อมูลผ่าน Model ด้วยคำสั่ง Blog::find(\$id)->update(\$data); และกลับหน้าเดิมด้วย return redirect()->back();

code-example.php

```
// app/Http/Controllers/AdminController.php
function change($id)
{
    $blog = Blog::find($id);
    $data = [
        'status' => !$blog->status
    ];
    Blog::find($id)->update($data);
    return redirect()->back();
}
```


การจัดกลุ่มเส้นทาง (Route Prefixes)

Route Prefixes (การจัดระเบียบกลุ่มเส้นทาง)

- ใช้สำหรับจัดระเบียบและแบ่งแยกพาร์ส URL ของแอปพลิเคชัน เช่น กลุ่ม URL หน้าจัดการระบบหลังบ้าน
- ช่วยลดความซ้ำซ้อนของการเขียนคำว่า /admin หรือ /api นำหน้าเส้นทางบ่อยๆ
- เรียกใช้งานผ่านเมธอด `Route::prefix('...')` เพื่อคลุมกลุ่มเส้นทางที่สอดคล้อง
- ช่วยเพิ่มความสวยงาม ความเป็นระบบ และลดความยาวของโค้ด Routing

code-example.php

```
// จัดระเบียบกลุ่ม URL ที่เริ่มต้นด้วย /admin
Route::prefix('admin')->group(function () {
    Route::get('/users', [UserController::class, 'index']); // URL คือ /admin/users
    Route::get('/settings', [SettingController::class, 'index']); // URL คือ /admin/settings
});
```

การจัดกลุ่มเส้นทางของนักเขียนบทความ

- จัดกลุ่ม แสดงเส้นทางโดยนักเขียนบทความ
- เช่น /author/blog

การแสดงผลเส้นทางสำหรับบุคคลทั่วไป

- แสดงเส้นทางบุคคลทั่วไป
- เช่น /blog

การจัดกลุ่มเส้นทางด้วย Route::prefix('author') ใน web.php

- เปิดไฟล์ routes/web.php
- จัดกลุ่มเส้นทางสำหรับนักเขียนบทความด้วยคำสั่ง Route::prefix('author')->group(...)
- กำหนดเส้นทางภายในกลุ่ม เช่น /about, /blog, /create และ /insert
- เส้นทางทั้งหมดในกลุ่มจะขึ้นต้นด้วย /author โดยอัตโนมัติ เช่น /author/blog หรือ /author/create

code-example.php

```
// routes/web.php
Route::prefix('author')->group(function () {
    Route::get('/about', [AdminController::class, 'about'])->name('about');
    Route::get('/blog', [AdminController::class, 'blog'])->name('blog');
    Route::get('/create', [AdminController::class, 'create'])->name('create');
    Route::post('/insert', [AdminController::class, 'insert']);
});
```

การปรับเส้นทาง Redirect หลังบันทึกข้อมูล (/author/blog)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- ปรับปรุงการเปลี่ยนเส้นทางในฟังก์ชัน insert ให้สอดคล้องกับ Route Prefix ใหม่
- เปลี่ยนจาก return redirect('/blog'); เป็น return redirect('/author/blog');
- เมื่อบันทึกบทความสำเร็จ ระบบจะนำผู้ใช้กลับไปยังหน้ารายการบทความของนักเขียน (/author/blog)

code-example.php

```
// app/Http/Controllers/AdminController.php
function insert(Request $request)
{
    $request->validate([
        [
            'title' => 'required|max:50',
            'content' => 'required',
        ],
        [
            'title.required' => 'กรุณาใส่ชื่อบทความ',
            'title.max' => 'ชื่อบทความต้องไม่เกิน 50 ตัวอักษร',
            'content.required' => 'กรุณาใส่เนื้อหา',
        ]
    ]);

    $data = [
```

การปรับเส้นทาง Redirect หลังอัปเดตข้อมูล (ฟังก์ชัน update)

- เปิดไฟล์ app/Http/Controllers/AdminController.php
- ปรับปรุงการเปลี่ยนเส้นทางในฟังก์ชัน update หลังบันทึกการแก้ไขข้อมูลเรียบร้อยแล้ว
- เปลี่ยนเส้นทาง Redirect ให้กลับไปหน้ารายการบทความของนักเขียน return redirect('/author/blog');
- เพื่อความสอดคล้องกับโครงสร้าง Route Prefix กลุ่ม author

code-example.php

```
// app/Http/Controllers/AdminController.php
function update(Request $request, $id)
{
    $request->validate([
        [
            'title' => 'required|max:50',
            'content' => 'required',
        ],
        [
            'title.required' => 'กรุณาใส่ชื่อบทความ',
            'title.max' => 'ชื่อบทความต้องไม่เกิน 50 ตัวอักษร',
            'content.required' => 'กรุณาใส่เนื้อหา',
        ]
    ]);

    $data = [
```

การปรับปรุงลิงก์ในเมนู Navbar (app.blade.php)

- เปิดไฟล์ resources/views/layouts/app.blade.php
- แก้ไขลิงก์ใน Dropdown เมนูของผู้ใช้งานเมื่อเข้าสู่ระบบแล้ว (@else)
- เปลี่ยนลิงก์เมนู เขียนบทความ เป็น href="/author/create"
- เปลี่ยนลิงก์เมนู บทความของฉัน เป็น href="/author/blog" เพื่อให้ทำงานร่วมกับ Route Prefix กลุ่ม author ได้อย่างถูกต้อง

code-example.html

```
<!-- resources/views/layouts/app.blade.php -->
<li class="nav-item dropdown">
  <a id="navbarDropdown" class="nav-link dropdown-toggle" href="#" role="button"
    สวัสดี , {{ Auth::user()->name }}
  </a>

  <div class="dropdown-menu dropdown-menu-end" aria-labelledby="navbarDropdown">
    <a class="dropdown-item" href="/author/create">เขียนบทความ</a>
    <a class="dropdown-item" href="/author/blog">บทความของฉัน</a>
    <a class="dropdown-item" href="{{ route('logout') }}"
      onclick="event.preventDefault();
        document.getElementById('logout-form').submit();"
      >ออกจากระบบ
    </a>
  </div>
</li>
```


การปรับปรุง Form Action ในแบบฟอร์มเขียนบทความ (form.blade.php)

- เปิดไฟล์ resources/views/form.blade.php
- แก้ไขแอตทริบิวต์ action ในแท็กฟอร์มให้ตรงกับเส้นทางบันทึกข้อมูลใหม่
- เปลี่ยนจาก action="/insert" เป็น action="/author/insert"
- เพื่อให้ข้อมูลจากแบบฟอร์มถูกส่งไปยัง Route POST /author/insert ภายใต้กลุ่ม author ได้อย่างถูกต้อง

code-example.html

```
<!-- resources/views/form.blade.php -->
@section('content')
    <h2 class="text text-center py-2">เขียนบทความใหม่</h2>
    <form method="POST" action="/author/insert">
        @csrf
        <div class="form-group">
            <label for="title">ข้อความ</label>
            <input type="text" name="title" class="form-control">
        </div>
        @error('title')
            <div class="my-2">
                <span class="text text-danger">{{ $message }}</span>
            </div>
        @enderror
    </form>
</section>
```

การปรับปรุงปุ่มลิงก์ในหน้าแดชบอร์ด (home.blade.php)

- เปิดไฟล์ resources/views/home.blade.php
- แก้ไขปุ่มลิงก์การทำงานในส่วนของ card-body
- เปลี่ยนลิงก์ เขียนบทความ เป็น href="/author/create"
- เปลี่ยนลิงก์ บทความทั้งหมด หรือ บทความของฉัน เป็น href="/author/blog" เพื่อให้เชื่อมต่อไปยังเส้นทางกลุ่ม author ได้อย่างถูกต้อง

code-example.html

```
<!-- resources/views/home.blade.php -->
<div class="card-body">
    @if (session('status'))
        <div class="alert alert-success" role="alert">
            {{ session('status') }}
        </div>
    @endif

    {{ __('You are logged in!') }}
    <br>
    <a href="/author/create" class="btn btn-primary">เขียนบทความ</a>
    <a href="/author/blog" class="btn btn-primary">บทความทั้งหมด</a>
</div>
```

การปรับปรุงปุ่มลิงก์กลับหน้ารวมบทความ (form.blade.php)

- เปิดไฟล์ resources/views/form.blade.php
- แก้ไขปุ่มลิงก์ บทความทั้งหมด ที่อยู่ด้านล่างของแบบฟอร์ม
- เปลี่ยนจาก href="/blog" เป็น href="/author/blog"
- เพื่อให้เมื่อคลิกปุ่ม จะนำผู้เขียนกลับไปยังหน้ารายการบทความของนักเขียน (/author/blog) ได้อย่างถูกต้อง

code-example.html

```
<!-- resources/views/form.blade.php -->
<div class="form-group">
  <label for="content">เนื้อหา</label>
  <textarea name="content" cols="30" rows="5" class="form-control"></t
</div>
@error('content')
  <div class="my-2">
    <span class="text text-danger">{{ $message }}</span>
  </div>
@enderror
<input type="submit" value="บันทึก" class="btn btn-primary my-3">
<a href="/author/blog" class="btn btn-success my-3">บทความทั้งหมด</a>
</form>
@endsection
```

การปรับปรุงปุ่มลิงก์กลับหน้ารวมบทความในหน้าแก้ไข (edit.blade.php)

- เปิดไฟล์ resources/views/edit.blade.php
- แก้ไขปุ่มลิงก์ บทความทั้งหมด ที่อยู่ด้านล่างของแบบฟอร์มแก้ไขบทความ
- เปลี่ยนจาก href="/blog" เป็น href="/author/blog"
- เพื่อให้เมื่อคลิกปุ่ม จะนำผู้เขียนกลับไปยังหน้ารายการบทความของนักเขียน (/author/blog) ได้อย่างถูกต้อง

code-example.html

```
<!-- resources/views/edit.blade.php -->
<div class="form-group">
  <label for="content">เนื้อหา</label>
  <textarea name="content" cols="30" rows="5" class="form-control"></t
</div>
@error('content')
  <div class="my-2">
    <span class="text text-danger">{{ $message }}</span>
  </div>
@enderror
<input type="submit" value="อัปเดต" class="btn btn-primary my-3">
<a href="/author/blog" class="btn btn-success my-3">บทความทั้งหมด</a>
</form>
@endsection
```

สัปดาห์ที่ 13 · การจัดการข้อมูลระดับโมเดลและคำนำหน้าเส้นทาง

01-406-093-203 LARAVEL COURSE

แบบทดสอบหลังเรียน (Post-test)

1. คำสั่งใดในการดึงข้อมูลของ Model เพื่อทำระบบแบ่งหน้า (Pagination) ใน Laravel?

A. `Blog::paginate(5);`B. `Blog::chunk(5);`C. `Blog::limit(5)->page();`D. `Blog::all()->split(5);`

3. การส่งลบแถวข้อมูลตามรหัส ID ผ่าน Eloquent Model ที่ถูกต้องคือข้อใด?

A. `Blog::find($id)->delete();`B. `Blog::delete($id);`C. `Blog::drop($id);`D. `Blog::remove($id);`

5. ตัวแปรใดใน Eloquent Model ที่ใช้ระบุฟิลด์ที่ 'ไม่อนุญาต' ให้เขียนผ่าน Mass Assignment (ตรงข้ามกับ \$fillable)?

A. `protected $guarded = [...];`B. `protected $hidden = [...];`C. `protected $locked = [...];`D. `protected $protected = [...];`

7. หากต้องการสร้างกลุ่มเส้นทางที่มีคำนำหน้า URL เป็น /author เช่น /author/blog หรือ /author/create ต้องใช้คำสั่งใดใน web.php?

A. `Route::prefix('author')->group(function () { ... });`B. `Route::group('author')->routes(function () { ... });`C. `Route::url('author')->group(function () { ... });`D. `Route::setPrefix('author')->group(function () { ... });`

9. ในหน้าแบบฟอร์มบันทึกบทความ (form.blade.php) แท็กฟอร์มควรมี action อย่างไรเมื่อส่งข้อมูลเข้า Route ได้กลุ่ม author?

A. `<form method="POST" action="/author/insert">`B. `<form method="POST" action="/insert">`C. `<form method="GET" action="/author/insert">`D. `<form method="POST" action="/author/create">`

2. การค้นหาข้อมูลแถวเดียวตามรหัส Primary Key ผ่าน Eloquent Model นิยมใช้บรรทัดใด?

A. `Blog::find($id);`B. `Blog::get($id);`C. `Blog::whereId($id);`D. `Blog::first($id);`

4. หากพบข้อผิดพลาด MassAssignmentException วิธีการแก้ไขที่ถูกต้องและปลอดภัยคือข้อใด?

A. เพิ่มรายชื่อฟิลด์คอลัมน์ในตัวแปร protected \$fillable ภายในไฟล์ Model (เช่น Blog.php)

B. ลบบรรทัด Validate ทั้งทั้งหมด

C. เปลี่ยนไปใช้ Query Builder แทน Model เสมอ

D. แก้โค้ดให้รับค่าผ่าน \$_POST โดยตรง

6. คำสั่งอัปเดตข้อมูลของบทความตามรหัส ID ผ่าน Eloquent Model ข้อใดถูกต้อง?

A. `Blog::find($id)->update($data);`B. `Blog::update($id, $data);`C. `Blog::save($id, $data);`D. `Blog::edit($id)->update($data);`

8. เมื่อย้าย Route มาอยู่ใต้ Prefix 'author' แล้วลิงก์ในเมนู Navbar (app.blade.php) สำหรับเขียนบทความและดูบทความของดิฉันควรเปลี่ยนเป็นข้อใด?

A. href="/author/create" และ href="/author/blog"

B. href="/create" และ href="/blog"

C. href="/author/add" และ href="/author/list"

D. href="/admin/create" และ href="/admin/blog"

10. คำสั่ง return redirect()->back(); ใน Controller ทำหน้าที่อะไร?

A. เปลี่ยนเส้นทางเบราว์เซอร์ให้กลับไปยังหน้าก่อนหน้าที่ผู้ใช้ส่งคำขอมา

B. ย้อนกลับประวัติฐานข้อมูล

C. ลบข้อมูลที่เพิ่งส่งมาล่าสุด

D. โหลดหน้ารีเฟรชแบบไม่ส่งพารามิเตอร์

คำถามท้ายบทเรียน

ตัวแปรใดในโมเดลของ Laravel ที่ใช้ทำหน้าที่ระบุรายชื่อฟิลด์คอลัมน์ที่อนุญาตให้เขียนผ่านระบบ Mass Assignment ได้?

A. protected \$allow

B. protected \$fillable

C. protected \$guarded

D. protected \$safety

คำอธิบาย: ตัวแปร protected \$fillable ใช้ระบุช่องข้อมูลความปลอดภัยที่อนุญาตให้กรอกบันทึกข้อมูลผ่านทางคำสั่ง create ได้ทีเดียว

Workshop 13: จัดการข้อมูลบทความด้วย Eloquent Model และจัดกลุ่มเส้นทางด้วย Route Prefix

- 1 เปลี่ยนการดึงข้อมูลและจัดการบทความใน AdminController จาก DB Query Builder มาเป็น Eloquent Model (Blog::paginate, Blog::insert, Blog::find)
- 2 กำหนดตัวแปร protected \$fillable = ['title', 'content', 'status']; ใน app/Models/Blog.php เพื่อป้องกัน MassAssignmentException
- 3 จัดกลุ่มเส้นทางของนักเขียนใน routes/web.php ด้วย Route::prefix('author')->group(...) สำหรับ /about, /blog, /create และ /insert
- 4 ปรับปรุงเส้นทาง redirect ใน AdminController และอัปเดตลิงก์ในหน้า View (app.blade.php, form.blade.php, home.blade.php, edit.blade.php) ให้ใช้ไปยังพารามิเตอร์ /author อย่างถูกต้อง

command_prompt.sh

```
// routes/web.php
Route::prefix('author')->group(function () {
    Route::get('/about', [AdminController::class, 'about'])->name('about');
    Route::get('/blog', [AdminController::class, 'blog'])->name('blog');
    Route::get('/create', [AdminController::class, 'create'])->name('create');
    Route::post('/insert', [AdminController::class, 'insert']);
});
```

ใบงานที่ 13: พัฒนาระบบจัดการบทความด้วย Eloquent Model, ป้องกัน Mass Assignment และจัดกลุ่ม Route Prefix

พัฒนาระบบบล็อกบทความโดยปรับปรุง Controller ให้ทำงานผ่าน Eloquent Model (Blog) ทั้งการแสดงผล การบันทึก การแก้ไข และการลบสถานะ พร้อมกำหนด \$fillable ใน Model เพื่อความปลอดภัย และจัดกลุ่มเส้นทางของนักเขียนทั้งหมดไว้ภายใต้ Route Prefix /author พร้อมปรับปรุงลิงก์ใน View และ Redirect ให้ทำงานได้อย่างสมบูรณ์

เกณฑ์การตรวจประเมินผล:	
การใช้งาน Eloquent Model แทน Query Builder ในการจัดการข้อมูลบทความ	30%
การกำหนด \$fillable ใน Model Blog.php เพื่อแก้ไขปัญหา MassAssignmentException	25%
การจัดกลุ่มเส้นทางด้วย Route::prefix('author') ใน routes/web.php ได้อย่างถูกต้อง	25%
การปรับปรุง Redirect และการเชื่อมโยงลิงก์ในหน้า View ต่างๆ ให้สอดคล้องกับพารามิเตอร์ใหม่	20%

